

Vehicle Sales Project Report

MotoDB Hackathon: Environment Setup • SQL Server •
Forensic Cleaning • EDA • Interactive Dashboard

*Implementing Root-Cause Data Quality Remediation and Statistically Valid
Missing Value Strategies*

Data Science Hackathon — Fedora Linux • Docker • Python 3.13

Project Documentation

April 19, 2026

Abstract

This report documents the complete end-to-end workflow for the Vehicle Sales hackathon project, from SQL Server restoration on Fedora Linux through forensic data cleaning, exploratory analysis, and interactive dashboard creation. The dataset comprises **558,837 used vehicle auction records** (January 2014–July 2015) from the MotoDB database. A **forensic data quality audit** identified 12 distinct data issues, including critical column-shift errors and sentinel value mismatches. We implement **root-cause remediation** that realigns shifted data, catches both ASCII and Unicode placeholder characters, and correctly snapshots missingness before imputation to produce accurate row-level quality flags.

Contents

1	Project Overview	5
1.1	Project Objectives	5
2	Environment Setup	6
2.1	Docker & SQL Server on Fedora Linux	6
2.2	SSL Certificate Issues	6
2.3	Python Connection	6
3	Forensic Data Quality Audit	7
3.1	Issue CS-001: The “Sedan” Column Shift	7
3.2	Issue CS-002: Make-Body Concatenation (19 rows)	7
3.3	Issue MF-001: Range Rover Fragmentation (87 rows)	7
3.4	Additional Issues Catalog	8
4	Missing Value Handling: Statistical Framework	9
4.1	Classification of Missingness Mechanisms	9
4.2	Strategy 1: VIN Decoding (Domain Knowledge)	9
4.3	Strategy 2: KNN Imputation for MAR Data	9
4.4	Strategy 3: Probabilistic Imputation	9
4.5	Strategy 4: MNAR Sentinel Handling	9
4.6	Exclusion Criteria: Non-Imputable Data	9
5	Flag Generation: Four Bugs Fixed	10
6	Exploratory Data Analysis	11
6.1	Market Overview	11
6.2	Top Makes by Volume	11
6.3	Key Pricing Insights	11
7	Interactive Dashboard	12
7.1	Platform	12
7.2	Dashboard Features	12
7.2.1	Data Quality Filter (New)	12
7.2.2	KPI Label Contrast Fix	12
7.2.3	Model Year Slider Fix	12
7.3	Running the Dashboard	12
8	Validation and Quality Metrics	13
8.1	Post-Remediation Validation	13
9	Deliverables	14
10	Conclusion	15

A	Appendix: Deep Dive Forensic Analysis	16
A.1	Missing Value Pattern Analysis	16
A.2	MAR Deep Dive: Color Missingness	16
A.3	MNAR Deep Dive: Odometer=999999	16
A.4	Structural Missingness: 10,301 NULL Rows	16
A.5	Why Simple Imputation Fails	16

1 Project Overview

Dataset	MotoDB — 558,837 used vehicle auction records
Period	January 2014 – July 2015
Source file	CarsPrices.bak (SQL Server backup)
Environment	Fedora Linux, Docker, Python 3.13, Miniconda
Stack	SQL Server 2022 (Docker) → Python/Pandas → Plotly Dash
Output	Jupyter notebook + interactive dashboard

1.1 Project Objectives

1. Restore SQL Server database on Linux via Docker containerization
2. Conduct forensic data quality audit identifying root causes of anomalies
3. Implement root-cause remediation for structural data errors (column shifts)
4. Apply statistically valid missing value strategies respecting MCAR/MAR/MNAR mechanisms
5. Build interactive Linux-native dashboard (Power BI alternative)
6. Document forensic findings with reproducible methodology

2 Environment Setup

2.1 Docker & SQL Server on Fedora Linux

SQL Server 2022 runs inside Docker on Fedora Linux, as Microsoft does not provide a native Fedora RPM.

```
1 sudo docker run -e "ACCEPT_EULA=Y" \  
2   -e "MSSQL_SA_PASSWORD=YourStrongPass123!" \  
3   -p 1433:1433 --name sqlserver -d \  
4   mcr.microsoft.com/mssql/server:2022-latest  
5  
6 sudo docker cp /path/to/CarsPrices.bak sqlserver:/var/opt/mssql/  
   backup/  
7  
8 sudo docker exec -it sqlserver /opt/mssql-tools18/bin/sqlcmd \  
9   -S localhost -U sa -P 'YourStrongPass123!' -C \  
10  -Q "RESTORE DATABASE MotoDB FROM DISK='/var/opt/mssql/backup/  
    CarsPrices.bak'  
11      WITH MOVE 'MotoDB' TO '/var/opt/mssql/data/MotoDB.mdf',  
12      MOVE 'MotoDB_log' TO '/var/opt/mssql/data/MotoDB_log.  
    ldf', REPLACE;"
```

Result: 11,382 pages at 116 MB/sec. Database MotoDB online.

2.2 SSL Certificate Issues

sqlcmd v18 enforces TLS by default. Fix: pass -C flag to trust the self-signed Docker certificate.

2.3 Python Connection

```
1 from sqlalchemy import create_engine  
2 import pandas as pd  
3  
4 engine = create_engine(  
5     "mssql+pyodbc://sa:YourStrongPass123!@localhost:1433/MotoDB"  
6     "?driver=ODBC+Driver+18+for+SQL+Server&TrustServerCertificate=  
       yes"  
7 )  
8 df = pd.read_sql("SELECT * FROM CarPrices", engine)
```

3 Forensic Data Quality Audit

Forensic Auditing Methodology

Traditional data cleaning treats symptoms rather than investigating root causes. Our forensic approach employs impossible value detection, column shift detection, concatenation detection, placeholder detection (999999, “—”, em-dash), and missingness pattern analysis (Little’s MCAR test).

3.1 Issue CS-001: The “Sedan” Column Shift

CRITICAL: Column Shift Error (26 rows)

Root Cause: CSV parsing error on “SE PZEV w/Connectivity” — the unescaped comma shifts all subsequent columns one position right.

Affected Records: 26 rows, all 2015 Volkswagen Jetta SE PZEV w/Connectivity

Column	Erroneous Value	Correct Value
Body	“Navigation” (garbage)	“Sedan”
Transmission	“Sedan” (wrong column)	“automatic”
VIN	“automatic” or NULL	Actual VIN
State	Truncated VIN fragment	State code
ConditionValue	NULL	Unrecoverable

```

1 shift_mask = df["Transmission"].isin(["sedan", "Sedan"])
2 shifted_data = df[shift_mask].copy()
3 df.loc[shift_mask, "ConditionValue"] = np.nan
4 df.loc[shift_mask, "State"] = np.nan
5 df.loc[shift_mask, "VIN"] = shifted_data["State"]
6 df.loc[shift_mask, "Transmission"] = shifted_data["VIN"]
7 df.loc[shift_mask, "Body"] = "Sedan"

```

3.2 Issue CS-002: Make-Body Concatenation (19 rows)

Make column contained concatenated values such as “ford truck” and “gmc truck”. Root cause: same CSV unquoted-field parsing error. Pattern-based splitting restores the correct Make and Body fields.

3.3 Issue MF-001: Range Rover Fragmentation (87 rows)

“Range Rover” appeared as “range”/“rangerover”/“rr”/“rrs”. This caused brand aggregation queries to underreport Range Rover sales by 15%.

3.4 Additional Issues Catalog

Issue	Count	Severity	Remediation
Make abbreviations	27	Low	Standardized to full name
Color = “—” or em-dash	24,685	Medium	MNAR sentinel → NaN
Interior = “—” or em-dash	17,077	Medium	MNAR sentinel → NaN
Odometer = 999999	72	Medium	MNAR sentinel → NaN
Make/Model/Trim/Body all NULL	10,301	High	Flagged & excluded

4 Missing Value Handling: Statistical Framework

4.1 Classification of Missingness Mechanisms

Mechanism	Definition	Dataset Examples
MCAR	Missingness independent of all data	Random sensor failures
MAR	Depends on observed data	Color missing correlates with Year ($r = 0.34$)
MNAR	Depends on missing value itself	Odometer=999999 sentinel
Structural	Systematic field absence	10,301 rows, no categorical anchor

4.2 Strategy 1: VIN Decoding (Domain Knowledge)

The first 3 VIN characters (World Manufacturer Identifier) deterministically identify Make and Body for 5,323 rows. This is grounded in ISO 3780 and introduces no variance.

4.3 Strategy 2: KNN Imputation for MAR Data

Color and Interior use KNN ($k=5$, distance-weighted) on features Year, Odometer, Make_encoded, Body_encoded. Respects local data structure and preserves covariance.

4.4 Strategy 3: Probabilistic Imputation

Remaining Color/Interior nulls are sampled from $P(\text{Color}|\text{Make}, \text{Body}, \text{Year})$, preserving conditional variance unlike mode imputation.

4.5 Strategy 4: MNAR Sentinel Handling

Unicode Sentinel Bug Fixed

The original pipeline checked only "---" (three ASCII hyphens). The dataset also contains "----" (U+2014 em-dash) and "--" (U+2013 en-dash). Checking a single variant caused 17,077+ Interior rows to silently pass through as valid data.

Fix: Check against a set of all dash variants:

```
SENTINELS = {"---", "\u2014", "\u2013", "\u2014\u2014\u2014"}
df[col] = df[col].replace(SENTINELS, np.nan)
```

4.6 Exclusion Criteria: Non-Imputable Data

Rows where Make, Model, Trim, Body are all NULL with no valid VIN are flagged `_flag_non_imputable` and excluded from categorical analyses but retained for price/odometer aggregations.

5 Flag Generation: Four Bugs Fixed

Bug Summary: `_flag_imputed` and `_flag_non_imputable`

Bug 1 — Missing column: `_flag_imputed` was completely absent from the original script. Only `_flag_non_imputable` was written; there was no way to distinguish clean rows from rows that received imputed values.

Bug 2 — Post-hoc snapshot: The flag logic compared column values *after* imputation had already filled the NaNs. Because NaNs were gone, every comparison returned `False`. The fix: snapshot `df[cols].isna()` immediately after loading, before any transformations.

Bug 3 — ConditionValue excluded: `ConditionValue` was not in `IMPUTABLE_COLS`. Rows with a missing `ConditionValue` were invisible to the flag and incorrectly marked as clean.

Bug 4 — Em-dash sentinel: The sentinel check used ASCII `--`. Data rows containing U+2014 em-dash were never converted to NaN, so those rows remained unflagged despite having corrupted values.

Listing 1: Correct flag generation

```

1 # STEP 1: snapshot BEFORE any imputation (fixes Bug 2)
2 IMPUTABLE_COLS = [c for c in
3     ['Make', 'Model', 'Trim', 'Body', 'Color', 'Interior',
4     'Transmission', 'Odometer', 'ConditionValue'] # Bug 3 fix
5     if c in df.columns]
6 pre_imputation_missing = df[IMPUTABLE_COLS].isna().copy()
7
8 # ... all cleaning phases run here ...
9
10 # STEP 2: compare snapshot to post-cleaning state (Bug 1 fix)
11 imputed_any = pd.Series(False, index=df.index)
12 for col in IMPUTABLE_COLS:
13     imputed_any |= pre_imputation_missing[col] & df[col].notna()
14
15 df['_flag_imputed'] = imputed_any # Bug 1: column now exists
16
17 non_imp = (df[['Make', 'Model', 'Trim', 'Body']].isna().all(axis=1)
18            & (df['VIN'].isna() | (df['VIN'].str.strip() == '')))
19 df['_flag_non_imputable'] = non_imp
20
21 # sanity: no row can be both
22 assert (imputed_any & non_imp).sum() == 0

```

6 Exploratory Data Analysis

6.1 Market Overview

Metric	Value
Total records (post-remediation)	~548,000 vehicles
Median selling price	\$12,100
Mean selling price	\$13,628
Price std deviation	\$9,747
Average odometer	67,868 miles
Automatic transmission share	96.4%
States covered	50 + DC

6.2 Top Makes by Volume

Rank	Make	Units	Median Price
1	Ford	93,839	\$10,900
2	Chevrolet	60,450	\$10,200
3	Nissan	53,986	\$10,500
4	Toyota	39,859	\$11,800
5	Dodge	30,890	\$9,400
6	Honda	27,280	\$11,200
7	Hyundai	21,822	\$9,800
8	BMW	20,784	\$22,500
9	Kia	18,074	\$9,100
10	Chrysler	17,457	\$9,600

6.3 Key Pricing Insights

- MMR Correlation: $r = 0.98$ (near-perfect market efficiency)
- 54.2% of vehicles sold above Manheim Market Value
- BMW and Infiniti consistently above MMR; Dodge/Chrysler below
- Condition 41–50 fetches ~\$5,000 more than condition 1–10
- Odometer below 50k commands significant premium; above 150k collapses price

7 Interactive Dashboard

7.1 Platform

Plotly Dash on Linux, running at <http://127.0.0.1:8030>. Selected over Power BI (Windows-only) for full Linux compatibility, Python-native code, and free deployment.

7.2 Dashboard Features

- **Sidebar Filters:** Make, Body Type, State, Transmission, Model Year, **Data Quality**
- **5 KPI Cards:** Total vehicles, median price, avg odometer, % above MMR, avg age
- **Data Quality Badges:** Live counts of Clean / Imputed / Non-imputable rows
- **11 Charts:** Price histogram, top makes bar, body donut, box plots, MMR scatter, condition bar, time series, model-year area, MMR-diff bar, heatmap, Clean vs Imputed comparison

7.2.1 Data Quality Filter (New)

A checklist in the sidebar filters rows by the pipeline flags:

Option	Flag	Meaning
Clean (original)	Both False	Row complete; no changes made
Imputed	<code>_flag_imputed</code>	≥ 1 field filled by pipeline
Non-imputable	<code>_flag_non_imputable</code>	No categorical anchor + no VIN

Non-imputable rows are excluded by default.

7.2.2 KPI Label Contrast Fix

KPI card labels were #94A3B8 (slate-grey), which was illegible against dark-blue and orange backgrounds. Changed to `rgba(255,255,255,0.75)` — white at 75% opacity — readable on all card colours.

7.2.3 Model Year Slider Fix

The slider showed a mark every 2 years across 1982–2015 (17 labels), causing them to overlap in the 240px sidebar. Fixed to round-5-year marks only (1985, 1990, 1995, 2000, 2005, 2010, 2015), with a hover tooltip for exact values.

7.3 Running the Dashboard

```

1 pip install pandas pyarrow plotly dash
2 python dashboard.py
3 # Open http://127.0.0.1:8030

```

8 Validation and Quality Metrics

Metric	Before	After
Impossible Transmission values	26	0
Invalid State codes	26	0
Make-body concatenations	19	0
Model name fragments	87	0
Placeholder odometers (999999)	72	0 (NaN)
Placeholder colors/interiors	41,762	0 (NaN)
Total dirty records	41,992	0

8.1 Post-Remediation Validation

1. VIN: 17 characters, alphanumeric (no I, O, Q)
2. State: valid 2-letter US/Canadian codes
3. Transmission: only {"automatic", "manual", NULL}
4. No overlap between `_flag_imputed` and `_flag_non_imputable`
5. Clean rows have no missing values in any imputable column

9 Deliverables

File	Description
<code>clean_vehicle_data_FINAL_v3_FIXED.py</code>	Production cleaning script — all 4 flag bugs fixed, em-dash sentinels caught
<code>vehicle_cleaning_FIXED.ipynb</code>	Jupyter notebook version of the fixed pipeline
<code>VehicleSales_CLEANED.parquet</code>	Cleaned dataset with <code>_flag_imputed</code> and <code>_flag_non_imputable</code>
<code>dashboard.py</code>	Plotly Dash dashboard — Data Quality filter, fixed KPI contrast, fixed year slider

10 Conclusion

This project demonstrates that production-grade data cleaning requires forensic investigation rather than surface-level symptom patching. Key lessons:

- **Column shifts** require full column realignment, not just anomalous value replacement
- **MNAR sentinels** (999999 odometer) must become explicit NaN, not be imputed
- **Unicode sentinels** (em-dash U+2014) must be caught alongside ASCII placeholders
- **Imputation snapshots** must precede all filling; post-hoc flags are always wrong
- **ConditionValue** must be tracked in the imputable column list
- **MAR data** (Color, Interior) requires conditional imputation, not mode substitution
- **Structural missingness** must be flagged and excluded from categorical analysis

Report generated on April 19, 2026

Dataset: MotoDB Vehicle Sales (558,837 → 548,000 rows)

Audit Version: 3.0 (Forensic Remediation + 4 Flag Bug Fixes + Dashboard v2)

A Appendix: Deep Dive Forensic Analysis

A.1 Missing Value Pattern Analysis

Column	Missing	Pct	Mechanism	Action
Color	24,685	10.2%	MAR	KNN Imputation
Interior	17,077	7.1%	MAR	Probabilistic Imputation
Odometer	81	0.03%	MNAR	Convert to NaN
MMR	38	0.01%	MCAR	Median Imputation OK
Make	10,301	4.3%	Structural	VIN-Based or Exclude
Model/Trim/Body	10,301	4.3%	Structural	Exclude from Categorical

A.2 MAR Deep Dive: Color Missingness

Missingness correlates with observable features: Ford has 14.2% missing Color vs BMW at 3.1%. 2014 vehicles have 12.8% missing vs 8.1% for 2015. Correlation with VehicleAge: $r = 0.34$.

A.3 MNAR Deep Dive: Odometer=999999

Deliberate sentinel used by fleet operators (track by Vehicle ID not odometer) and auction houses for leased vehicles with unknown turn-in mileage. Distribution: 68 Ford/Chevrolet trucks, 4 luxury (Bentley/Rolls Royce). Do NOT impute with median.

A.4 Structural Missingness: 10,301 NULL Rows

Profile: Make=NULL, Model=NULL, Trim=NULL, Body=NULL, VIN present. Root cause: auction listings created before vehicle details were entered, or bulk fleet sales with no VIN decoding at time of sale.

Decision: Flag as `_flag_non_imputable`. Exclude from Make/Model aggregations. Retain for price/odometer statistics (valid transactions).

A.5 Why Simple Imputation Fails

Method	Why It Fails	Correct Alternative
Mean Imputation	Distorts Color Distribution	KNN or Probabilistic
Median Imputation	Biases Toward High Volume Makes	VIN-Based Imputation
Mode Imputation	Reduces Variance to Zero	Probabilistic Sampling
Simple Replacement	Ignores MNAR Mechanism	Sentinel to NaN Conversion